

Python: Introduction, Syntax & Data Types

Compilation: Yogesh Haribhau Kulkarni

Introduction, Syntax & Data Types

Python: Quick Introduction

Guess

What are the differences with the programming languages you know?

```
x = 34 - 23
y = "Hello"
z = 3.45
if z == 3.45 or y == "Hello":
    x = x + 1
    y = y + " World"
print(x)
print(y)
```

Why Python?

- Readability
- Ease of use
- “Fits in your head”
- Incremental sense of accomplishment, aka “gets things done”
- Good libraries
- Deployment, aka “Lookie what I did!”

Truths about Good Programmers

- Lazy (in a good way)
- Just want things to work
- Spoiled kids who just want to have fun
- And sometimes create Fortune 100 companies

One Truth About Python

- Power scales with the ability of the programmer
- Novices can do simple things
- Really bright people build tools
- Novices leverage these tools
- Lone sys-admins <3 perl
- Mavericks in small work-groups <3 Python

Brief History of Python

- Invented in the Netherlands, early 90s by Guido van Rossum
- Named after Monty Python (a British comedy group, the language has a playful approach)
- Open sourced from the beginning
- Considered a scripting language, but is much more
- Used by Google from the beginning
- Increasingly popular

Overview

Introduction

- Python is a simple, yet powerful interpreted language.
- Numerous libraries: NumPy, SciPy, Matplotlib
- Named after Monty Python.
- Open Source and Free
- Invented by Guido van Rossum.

Python's Benevolent Dictator For Life

“Python is an experiment in how much freedom programmers need. Too much freedom and nobody can read another's code; too little and expressiveness is endangered.”

- Guido van Rossum



(Reference:
https://en.wikipedia.org/wiki/Guido_van_Rossum)

What is Python?

- Interpreted
- Object-oriented
- High-level
- Dynamic semantics
- Cross-platform
- Readability.

Compiled Languages

- Needs entire program
- Translates directly to machine codes
- Exe native and fast
- Usually statically typed
- Types known during compilation
- Change in type : recompilation
- Ideal for compute-heavy tasks
- E.g. C, C++, FORTRAN

Interpreted Languages

- Interpreted on the fly
- No need to compile: can execute right away
- Usually dynamically typed
- Non-syntax errors are detected only in run-time
- Slower than compiled languages
- Ideal for small tasks
- E.g. Python, Perl, PHP, Bash

Note: Python, at the beginning, loosely checks the program. Only at run time, line-by-line, it checks for errors. So, if the error statements are not in the running path, their error does not get reported. So, it's a bit relaxed. That's the objection for its use in production code, where strong type checking and error checking, upfront is essential.

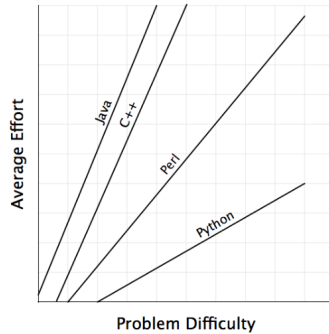
JIT-Compiled Languages

- Between compiled and interpreted
- Code is initially interpreted, hotspots compiled
- Deduce types during compilation, can change in run-time
- Slower than compiled
- E.g. Java, C#

“C” guys to take pride in

- Python interpreter written in “C”
- Source code at www.python.org
- So, Python Interpreter is compiled exe

Completely Controversial Observations about Languages



Python Help

- Home page – <http://www.python.org>
- Wiki – <http://wiki.python.org/>
- Packages – <https://pypi.python.org/pypi>
- Projects at <http://sourceforge.net> and [github.org](https://github.com)

Important features

- Built-in high level data types: strings, lists, dictionaries, etc.
- Usual control structures: if, if-else, if-elif-else, while, for
- Levels of organization: functions, classes, modules, packages
- Extensions in C and C++ possible

Python 2 vs Python 3

- Python 2.7 reached **end-of-life on January 1, 2020** — do not use it for new code.
- **Python 3** (current stable: 3.12+) is the only supported version.
- This workshop uses **Python 3** throughout.

Quick Check: Introduction

Think About It

Python is called an “interpreted” language, yet the CPython interpreter itself is a compiled C program. Does that make the compiled-vs-interpreted distinction a property of the language, or of something else?

Answer

It’s a property of the *implementation*, not the language. Python the language has no built-in notion of how it must

be run – CPython interprets it, but PyPy JIT-compiles it, and tools like Cython compile it to machine code. “Python is interpreted” really means “the standard CPython tool interprets it,” not that the language itself demands interpretation.

Installations

Installing Python

- Pre-installed on most Unix, Linux and MAC OS X
- Windows: Python.org or Anaconda (preferred)
- Anaconda has necessary libraries
- 2.* and 3.* both
- ‘pip’ and ‘conda’

Installing Python via Anaconda Distribution

- Download the latest **Python 3** Anaconda from <https://www.anaconda.com/download>
- Choose the 64-bit installer for your OS (Windows / macOS / Linux)
- Includes conda package manager and all essential scientific libraries
- After install, update: `conda update conda && conda update --all`
- Lightweight alternative: **Miniconda** — installs only Python + conda

The Python shell, I

- Can run from “shell”, IDE, Notebook
- Shell/Command Line (below):
- Start typing at `>>>`

```
$ python

Python 3.5.3 | packaged by conda-forge | (default,
May 12 2017, 16:16:49) [MSC v.1900 64 bit
(AMD64)] on win32
Type "help", "copyright", "credits" or "license"
for more information.
>>>
```

First Python Error

- Type plain English and Python rejects it — it needs valid syntax.

```
>>> I come in peace, please take me to your leader
File "<stdin>", line 1
I come in peace, please take me to your leader
~
SyntaxError : invalid syntax
>>>
```

The Python shell, II

- Expressions: evaluated, result printed:

```
>>> 2+2
4
```

- Line continuation

```
>>> "hello" + \
... " world!"
'hello world!'
```

- Prompt changes to ‘...’ on continuation lines.

The Python shell, II

- Syntactically correct:

```
>>> print('I come in peace, please take me')
```

- `print` is a function, with defined syntax.
- If you don’t obey:

```
>>> print 'I come in peace, please take me'
File "<stdin>", line 1
print 'I come in peace, please take me'
~
SyntaxError: Missing parentheses in call to
'print'
```

The Python shell, II

- To exit:

```
>>> good-bye
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
NameError name 'good' is not defined
```

```
>>> if you don't mind, I need to leave
File "<stdin>", line 1
if you don't mind, I need to leave
~
SyntaxError: invalid syntax
```

- Better to:

```
>>> quit()
```

Exercises

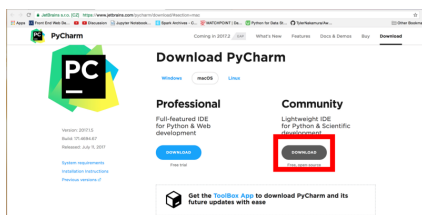
- Find more about JIT-compilation.
- Install any IDE, having debugging.
- Get familiar with command prompt.

Integrated Dev Env't

- **VS Code** <https://code.visualstudio.com/> — free, lightweight, excellent Python extension
- **PyCharm** <https://www.jetbrains.com/pycharm/> — full-featured, Community edition is free
- **Spyder** <https://www.spyder-ide.org/> — MATLAB-like, bundled with Anaconda
- **IDLE** — minimal IDE that comes with python.org distribution

PyCharm

Download the community edition of Pycharm



PyCharm Project

Create a New Project



PyCharm Exercise

- Create a directory.
- Create New Project in Pycharm.
- Give the newly created dir as path.
- Create New Python file. Write:

```
print("Hello World!!")
```

- Run.
- Result?.
- Get familiar with Pycharm UI.

Quick Check: Getting Started

Think About It

Typing an English sentence at the >>> prompt gives a **SyntaxError** instead of Python just printing it back or ignoring it. Why?

Answer

The shell doesn't treat what you type as plain text by default — every line is parsed as executable code first. Unquoted words are assumed to be part of a statement, not data, so Python tries (and fails) to make sense of them as a program. This is also why `print 'hi'` (no parentheses) fails in Python 3: `print` is a function, and a function call needs valid call syntax before Python will run anything at all.

What is Python?

- Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.
- Python is simple and easy to learn, open source, free and cross-platform.
- Python provides high-level built-in data structures.
- Python is useful for rapid application development, as a scripting or glue language.
- Python emphasizes readability, and supports modules and packages.

The Python Shell

- Python can be run from a shell, IDE, or notebook — commands are entered at the >>> prompt.
- Expressions are evaluated and the result is printed immediately.
- The prompt changes to ... on continuation lines (loops, function definitions, etc.)

```
> python
Python 3.11.0 (default) [MSC v.1900 64 bit
(AMD64)] on win32
>>> 2 + 2
4
>>> "hello" + \
...     " world!"
'hello world!'
```

Lines

- Statement separator: a semicolon.
- Only needed for multiple statements on one line — otherwise, nothing.
- Continuation: a backslash.
- But not necessary inside a “context”: (), { }, [].

Blocks and Indentation

- Blocks are formed by indentation — no begin/end brackets.
- Indentation is 4 spaces, no hard tabs.
- `if`, `while`, `def`, `except`, etc. end with a colon.
- A colon must be followed by an indented block on the next line.
- One script can be run directly, or imported by other modules.

Syntax

Comments

Everything after # is ignored – no block comments in Python.

```
# compute the percentage of the hour that has elapsed
percentage = (minute * 100) / 60

v = 5           # useless/redundant comment
v = 5           # velocity in meters/second -- useful comment
```

Use a comment to temporarily “disable” a line of code, rather than deleting it.

f-Strings: Modern String Formatting (Python 3.6+)

Embed variables and expressions directly in strings by prefixing with f. f-strings are **faster**, **more readable**, and the **recommended** style for all new Python 3 code.

```
name = "Alice"
age = 30

# Old %-style (avoid in new code)
print("Hello, %s. You are %d." % (name, age))
# .format() style
print("Hello, {}. You are {}".format(name, age))
# f-string (preferred since Python 3.6)
print(f"Hello, {name}. You are {age}.")
print(f"Next year: {age + 1}")      # expressions work too
print(f"Pi: {3.14159:.4f}")        # format specs work too
```

Assignment

- Assignment creates references, not values: `tmp = "hello"; tmp = 10` – the string gets deallocated.
- Augmented assignment is valid: `x += 1`.
- Pre/post increment/decrement (`x++`, `++x`) are **not** valid Python – there is no `++` operator.
- Multiple assignment to one object: `x = y = z = 1`.
- Tuple unpacking: `(x, y, z) = (3.5, 5.5, 'string')`.
- Swapping values: `(x, y) = (y, x)`.

Built-in Object Types

- Numbers: 3.1415, 1234, 999, 3+4j
- Strings: `'spam'`, `"guido's"`
- Lists: [1, [2, 'three'], 4]

- Dictionaries: `{'food': 'spam', 'taste': 'yum'}`
- Tuples: (1, 'spam', 4, 'U')
- Sets: {1, 2, 3, 'foo', 'bar'}

Numbers

- Integers (unlimited precision): 1234, -24, 0, 999999999999
- Float: 3.1415, 2.7122
- Octal and hex: 0o177, 0x9ff
- Complex: 3+4j, 3.0+4.0j, 3j

Strings (immutable sequences)

- single quote `s1 = 'egg'`, double quotes `s2 = "spam's"`, triple quotes block `"""..."""`
- concatenate `s1 + s2`, repeat `s2 * 3`
- index, slice `s2[i]`, `s2[i:j]`, length `len(s2)`
- formatting `"a {} parrot".format('dead')`
- iteration `for x in s2`, membership `'m' in s2`

Lists

- Ordered collections of arbitrary objects, accessed by offset.
- Variable length, heterogeneous, arbitrarily nest-able.
- Mutable – an array of object references.

```
L = []           # empty list
L2 = [0, 1, 2, 3] # four items
L3 = ['abc', ['def', 'ghi']] # nested
L2[i], L3[i][j]   # index
L2[i:j], len(L2)  # slice, length
L1 + L2, L2 * 3   # concatenate, repeat
L2.append(4); L2.sort(); L2.reverse()
del L2[k]; L2[i] = 1; L2[i:j] = [4, 5, 6]
```

Dictionaries

- Accessed by key, not offset – unordered collections of arbitrary objects.
- Variable length, heterogeneous, arbitrarily nest-able, mutable.
- Implemented as hash tables of object references.

```
d1 = {}           # empty
d2 = {'spam': 2, 'eggs': 3} # two items
d3 = {'food': {'ham': 1, 'egg': 2}} # nested
d2['eggs'], d3['food']['ham'] # indexing
d2.keys(), d2.values(), len(d1)
d2[key] = new      # add/change
del d2[key]         # delete
```

Tuples and Files

- **Tuples** are like lists but immutable – use them when the content shouldn't change.
- **Files**:

```
f = open('data', 'r')
s = f.read()       # read all
s = f.read(n)      # read n bytes
s = f.readline()   # read next line
lines = f.readlines() # read into a list
out = open('/tmp/spam', 'w')
out.write(s); out.writelines(lines); out.close()
```

Control Flow: if, while, for

```
if not done and (x > 1):
    doit()
elif done and (x <= 1):
    dothis()
else:
    dothat()

while True:
    line = read_line()
    if len(line) == 0:
        break

for letter in 'hello world':
    print(letter)
for i in range(2, 10, 2):
    print(i)
```

Functions

- Functions can return any type of object – `None` is returned by default.
- Multiple values can be returned; parameters can have default arguments.
- Variable-length arguments and anonymous (`lambda`) functions are supported.

```
def test(a, b=2, d=some_func):
    return d(a, b)

test(3)
test(b=4, a=3)
test(1, 2, lambda x, y: x * y)
```

Quick Check: Syntax

Think About It

C and Java use braces { } to mark blocks – indentation is just a human-readable convention on top of that. Python uses indentation itself as the block boundary. What's the tradeoff?

Answer

It forces what the code *looks* like and what it *does* to always agree – you can't have a block that visually appears nested one way but executes another, a class of bug that misleading brace placement can cause in C or Java. The tradeoff is that whitespace becomes meaningful: mixing tabs and spaces, or one stray space, can silently change what the code does, without an obvious visual cue like a misplaced brace.

Data Types

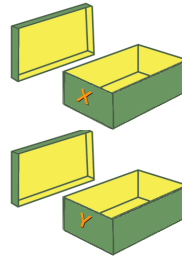
Variables

- A variable can be seen as a label (or name tag) attached to a value.
- While the program is running, variables are accessed and sometimes changed, i.e. a new value will be assigned to a variable.
- In Python, the **value** has a type — the variable itself does not.
- The same variable name can point to an integer now and a string later.

Variables in C/C++ or Java

```
int x;
int y;
```

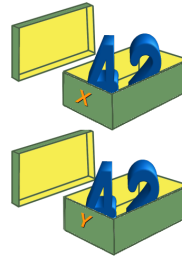
- Such declarations make sure that the program reserves memory for two variables with the names x and y.
- The variable names stand for the memory location.
- It's like the two empty shoeboxes, labelled with x and y.
- Like the two shoeboxes, the memory is empty as well.



Variables in C/C++ or Java

```
x = 42;
y = 42;
```

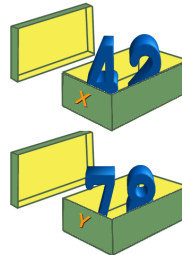
- Putting values into the variables can be realized with assignments.
- Two numbers are physically saved in the memory, which correspond to the two shoeboxes.



Variables in C/C++ or Java

```
y = 78;
```

- If we assign a new value to one of the variables, let's say the values 78 to y.
- We have exchanged the content of the memory location of y.



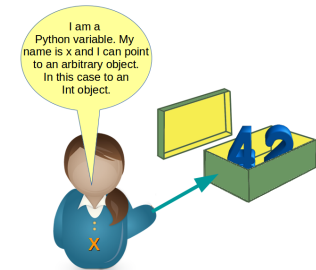
Variables in Python

- There is no declaration of variables required.
- Not only the value of a variable may change during program execution but the type as well.
- You can assign an integer value to a variable, use it as an integer for a while and then assign a string to the same variable.

```
i = 42          # data type is implicitly set to
                integer
i = 42 + 0.11    # data type is changed to float
i = "forty"     # and now it will be a string
```

All variables in Python are references

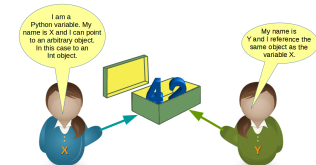
Python variables are references to objects, but the actual data is contained in the objects:



As variables are pointing to objects and objects can be of arbitrary data type, variables cannot have types associated with them.

All variables are references

```
>>> x = 42
>>> y = x
```

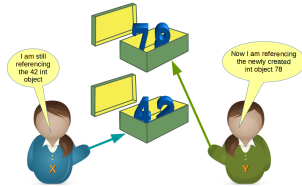


All variables are references

What will happen, when we execute

```
y = 78
```

Python will create a new integer object with the content 78 and then the variable `y` will reference this newly created object, as we can see in the following picture:

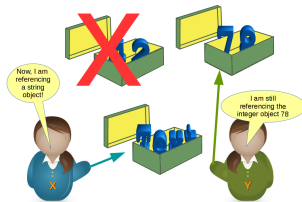


All variables are references

What will happen, when we execute

```
x = "Text"
```

The previously integer object "42" will be orphaned after this assignment. It will be removed by Python, because no other variable is referencing it.



All variables are references

In Python, **all objects are ever passed by reference**. In particular, **variables always store a reference to an object**, never a copy!

Hence, you have to be careful when modifying objects:

```
>>> a = [1,2,3]
>>> b = a
>>> b.remove(2)
>>> print(a)
```

???

Q: How many items are in the `a` list now?

All variables are references

However

```
>>> a = 1
>>> b = a
>>> b += 1
>>> a
1
>>> b
2
```

How can you explain this?

The `b += 1` operator could be replaced by `b = b + 1`, and the `b + 1` expression yields a new value.

Dynamic Typing: Python

- Variables come into existence at first assignment.
- A variable can refer to an object of any type.
- Can switch type based on assignment
- All types (almost) treated the same way.
- Type errors only caught in runtime.

Variables

Definition-Declaration?

- Assignment: Definition + Declaration:

```
>>> n = 17
>>> pi = 3.1415926535897931
>>> message = 'And now for something
different'
```

Variables Value

- Display value

```
>>> print(n)
17
>>> print(pi)
3.141592653589793
>>> print(message)
???
```

Variables Type

- Display type

```
>>> print(type(n))
<class 'int'>
>>> print(type(pi))
<class 'float'>
>>> print(type(message))
??
```

Variable Names

- Can be arbitrarily long.
- Can contain both letters and numbers,
- Cannot start with a number.
- `(.)` can appear in a name.
- Reserves 33 keywords

Variable Names Errors

```
>>> 76trombones = 'big parade'
SyntaxError: invalid syntax
>>> more@ = 1000000
SyntaxError: invalid syntax
>>> class = 'Advanced Theoretical Zymurgy'
SyntaxError: invalid syntax
```

Choosing mnemonic variable names

Variable names different but accomplishment same.

- ```
a = 35.0
b = 12.50
c = a * b
print(c)
```
- ```
hours = 35.0
rate = 12.50
pay = hours * rate
print(pay)
```
- ```
x1q3z9ahd = 35.0
x1q3z9afd = 12.50
x1q3p9afd = x1q3z9ahd * x1q3z9afd
print(x1q3p9afd)
```



Which one better?

## Basic Data types

- **bool** : True, False.
- **int** Integer numbers, arbitrary precision (no separate long type in Python 3).
- **float** Double precision floating-point numbers
- **str** Unicode text (immutable sequence of characters).
- **list** Mutable list
- **tuple** Immutable list
- **set** Unordered collection
- **dict** Key/value mapping

## Built-in object types

- Numbers : 3.1415, 1234, 999, 3+4j
- Strings : 'spam', "guido's"
- Lists : [1, [2, 'three'], 4]
- Dictionaries : {'food': 'spam', 'taste': 'yum'}
- Tuples : (1, 'spam', 4, 'U')
- Sets: {1,2,3, 'foo', 'bar'}

## Numbers

- Integers : 1234, -24, 0
- Unlimited precision integers : 999999999999
- Float : 3.1415, 2.7122
- Oct and hex :0o177, 0x9ff
- Complex : 3+4j, 3.0+4.0j, 3J

## Integer

```
int_1 = 1
int_2 = -2
int_3 = 100

print(int_1 + int_2 + int_3) # for integers, + is
 "plus"

this is not what it looks like
print(1,000,000)

eh, what is this then?
i = 1,000,000
type(i)

Can I add these? Yes, but...
i = 1,000,000
j = 2,000,000
```

```
i + j
i / j
```

## Floating Point Numbers

Decimal numbers, representations of fractions, "real-valued"

```
float_1 = 1.2
float_2 = -4.0
float_3 = 10.0

print(float_1 + float_2 + float_3)

males = 7
females = 10
fraction = males/(males+females)
print("Percentage men: {}".format(fraction))
```

## Booleans

True or False.

```
bool_1 = True
bool_2 = False

print(bool_1)
print(bool_2)
```

## String Quotes

- Single and double quotes interchangeable:

```
>>> "a string" == 'a string'
True
```

- One inside another ok:

```
>>> a = "Isn't it ok?"
>>> b = 'Yes, he said.'
```

- Just be consistent.

## Multi Line

Multi-line strings with three quotes.

```
str_10 = """
(CNN)AirAsia Flight QZ8501 climbed rapidly before
it crashed, a top Indonesian official said
Tuesday, according to The Jakarta Post. Then
the plane stalled, Transportation Minister
Ignasius Jonan said at a parliamentary
```

```
hearing, according to the AFP and Reuters
news agencies. "The plane, during the last
minutes, went up faster than normal speed ...
after then, it stalled. That is according to
the data from the radar," Jonan said,
according to the news agencies.
"""
print(str_10)
```

## String Indexing

Can be indexed (subscripted), start at 0. No separate character type: a string of size one.

```
word = 'Python'
word[0] # character in position 0
word[1]
word[5] # character in position 5
word[-1] # last character
word[-2] # second-last character
word[-6] #?
```

## Slicing

Slice direction is always left to right.

## Intuition

Think of index positions as marks *between* characters, not the characters themselves: `word[a:b]` means "everything from mark a up to (but not including) mark b." Negative indices count backwards from the end, so `word[-3:]` always means "the last 3 characters," whatever the string's length.

```
word[0:2] # from position 0 (included) to 2
 (excluded)
word[2:5] # from position 2 (included) to 5
 (excluded)
word[2:] # from position 2 (included) to the end
word[:3] # from beginning to position 3 (excluded)
word[-3:] # last three characters
word[-3:-1] # penultimate two characters
word[-1:-3] # Error: wrong direction
```

## String Exercise

- Assign the string 'Dealing with Data' to a Python variable.
- Show usages positive indexing/slicing
- Show usage of negative indexing/slicing

## Start-End

Return `True` if `t` is the initial/final substring

```
s.startswith(t)

s.endswith(t)
```

## String Exercise

- Write a Python program to get a new string from a given string where “Is” has been added to the front. If the given string already begins with “Is” then return the string unchanged.
- Write a Python program to get the `n` (non-negative integer) copies of the first 2 characters of a given string. Return the `n` copies of the whole string if the length is less than 2

## Cases

A copy of `s` is made, modified and returned.

```
s.capitalize() # First letter of sentence
uppercase

s.title() # First letter of all the words
uppercase

s.lower() # All letters lowercase

s.upper() # All letters uppercase
```

## Split

Split `s` at every occurrence of `t` and return a list of parts. If `t` is omitted, split on whitespace.

```
s.split(t)
```

## Exercise

- Write a program that computes the net amount of a bank account based a transaction log from console input. The transaction log format is shown as following:

```
D 100
W 200
```

- D means deposit while W means withdrawal.

- Suppose the following input is supplied to the program:

```
D 300
D 300
W 200
D 100
```

- Then, the output should be: 500
- Hints: In case of input data being supplied to the question, it should be assumed to be a console input.

## Solution

```
import sys
netAmount = 0
while True:
 s = input("Enter the transaction")
 if not s:
 break
 values = s.split(" ")
 operation = values[0]
 amount = int(values[1])
 if operation=="D":
 netAmount+=amount
 elif operation=="W":
 netAmount-=amount
 else:
 pass
print(netAmount)
```

## Replace

Return a *copy* of `s` with all old replaced by new.

```
s.replace(old, new)
```

## Strip

Return a *copy* of with the leading (resp. trailing, resp. leading and trailing) whitespace removed.

```
s.lstrip()

s.rstrip()

s.strip()
```

## Finding text within string variables

```
word = "And on and on and on and on..."
ind = word.find("on")
print(ind)
print("The first time on is at position",
 word.find("on"))
```

```
first_appearance = word.find("on")
second_appearance =
 word.find("on",first_appearance+1)
print("The second time on is at ",
 second_appearance)
```

## Finding text within string variables

Looking for “on” at the second half of “word”

```
midpoint = int(len(word)/2) # finds middle of
string word
second_half_appearance = word.find("on",midpoint)
print("First time 'on' in the second half: ",
 second_half_appearance)
```

## Finding text within string variables

Exercise: Convert the code below to make it case-insensitive.

```
word = "Python: And on and on and on and on..."
lookfor = "PYTHON"
count = word.count(lookfor)
print("See ", lookfor, " that many times: ",
 count)
```

## Concatenating strings

(Ref: Transforming Code into Beautiful, Idiomatic Python - Raymond Hettinger)

Old way:

```
names = ['raymond', 'rachel', 'matthew', 'roger',
 'betty', 'melissa', 'judith', 'charlie']

s = names[0]
for name in names[1:]:
 s += ', ' + name
print(s)
```

Better:

```
print(', '.join(names))
```



## Joining list of Strings

```
mystring1 = "practical data science"
mylist1 = mystring1.split(" ")
print(mystring1)
print(mylist1)

" ".join(mylist1) # Try different "<c>" here
```

## String Comparisons

```
str_1 = "hello"
print("equality:")
print(str_1 == "hello")

print(str_1 == "Hello")
```

## String Exercise

- Consider the string “billgates@microsoft.com”.
- Write code that finds the username of the email address and the domain of the email address.
- Use multiple methods (eg. slicing with raw numbers, slicing with find, and the split command).

## String Formatting

- To embed other information into strings.
- Sometimes with special formatting constraints.

```
print('Coordinates: {}, {}'.format('37.24N',
 '115.81W'))

print('Coordinates: {0}, {1}'.format('37.24N',
 '115.81W'))

lon = '37.24N'
lat = '115.81W'
print('Coordinates: {0}, {1}'.format(lon, lat)) #
 skipping 0,1 works?

print('Latitude: {0}, Longitude: {1} ==> [{0},
 {1}].format('37.24N', '115.81W'))

Skipping 0,1 does not work as, then, it expects
 4 arguments.
```

## String Formatting

Alternatively, instead of using the 1, 2, etc. format, specify names for the attributes:

```
print('Coordinates: {latitude}, {longitude}'
 .format(latitude='37.24N',
 longitude='115.81W'))
```

## strings (recap)

- single quote: `s1 = 'egg'`
- double quotes: `s2 = "spam's"`
- triple quotes: `block = """..."""`
- concatenate: `s1 + s2`
- repeat: `s2 * 3`
- index,slice: `s2[i], s2[i:j]`
- length: `len(s2)`
- formatting: `"a {} parrot".format('dead')`

## Exercise

- Write a Python program to swap two string variables
- Write a Python program to check if a string is numeric.
- Write a Python program to check if lowercase letters exist in a string.
- Write a Python program to check if multiple variables have the same value

## Exercises

Write a Python program to get a string made of the first 2 and the last 2 chars from a given string. If the string length is less than 2, return instead of the empty string.

Sample String : 'w3resource'

Expected Result : 'w3ce'

Sample String : 'w3'

Expected Result : 'w3w3'

Sample String : 'w'

Expected Result : Empty String

## Exercises

Write a Python program to get a string from a given string where all occurrences of its first char have been changed to '\$', except the first char itself.

Sample String : 'restart'

Expected Result : 'resta\$t'

## Exercises

Write a Python program to get a single string from two given strings, separated by a space and swap the first two characters of each string.

Sample String : 'abc', 'xyz'

Expected Result : 'xyc abz'

## Exercises

Write a Python program to add 'ing' at the end of a given string (length should be at least 3). If the given string already ends with 'ing' then add 'ly' instead. If the string length of the given string is less than 3, leave it unchanged.

Sample String : 'abc'

Expected Result : 'abcing'

Sample String : 'string'

Expected Result : 'stringly'

## Exercises

Write a Python program to find the first appearance of the substring 'not' and 'poor' from a given string, if 'bad' follows the 'poor', replace the whole 'not'...'poor' substring with 'good'. Return the resulting string.

Sample String : 'The lyrics is not that poor!'

Expected Result : 'The lyrics is good!'

## Quick Check: Data Types

### Think About It

After `i = 42` then `i = "forty"`, has the variable `i` changed its type, or has something else happened?

### Answer

Something else: `i` never had a type to begin with. Variables in Python are just names pointing to objects; `i = 42` makes the name `i` point to an integer object, and `i = "forty"` makes the same name point to a completely different string object. The old integer object is unchanged (and gets garbage-collected once nothing references it) – only the *name* moved. This is what “dynamically typed” means: types belong to values, not to variable names.

## Operators

## Operators

- The usual unary and binary arithmetic operators: +, −, \*, /, \*\*, <, >, etc.
- Logical operators: and, or, not.
- Numerical and string comparison: <, >, <=, ==, !=, ...
- Order of operations: PEMDAS: Parentheses, Exponentiation, Multiplication, Division, Addition and Subtraction.
- Operators with the same precedence are evaluated from left to right.

## First exercise

Calculate:

How much is  $2^{38}$  ?

## Operators

Some operators are defined for non-numeric types:

```
>>> "U" + 'ZH'
'UZH'
```

Some support operands of mixed type:

```
>>> "a" * 2
'aa'
>>> 2 * "a"
'aa'
```

Some do not:

```
>>> "aaa" / 3
Traceback (most recent call last):
 File "<stdin>", line 1, in <module>
TypeError: unsupported operand type(s) for /:
'str' and 'int'
```

## Operators

There are two kinds of division operators:

- "true division" performed by "/"
- "floor division" performed by "//"

```
>>> 10 / 3
3.3333333333333335
>>> 10.0 / 3.0
3.3333333333333335
>>> 10.5 / 3.5
3.0
>>>
>>> 9 // 3
```

```
3
>>> 10 // 3
3
>>> 11 // 3
3
>>> 12 // 3
4
>>> 10.0 // 3
3.0
```

## Operators

The "%" operator computes the remainder of integer division.

```
>>> remainder = 7 % 3
>>> print(remainder)
1
```

## Operators

The + operator works with strings, but it is not addition in the mathematical sense. Instead it performs concatenation.

```
>>> first = 10
>>> second = 15
>>> print(first+second)
25
>>> first = '100'
>>> second = '150'
>>> print(first + second)
100150
```

## Operators

Use assignment '=' statement, with shortcut notations for compound updates:

$a += b$  short for  $a = a + b$ ,  
 $a -= b$  short for  $a = a - b$ ,  
 $a *= b$  short for  $a = a * b$ ,

```
>>> a = 1
>>> print(a)
1
```

## The is operator

Tests whether two names refer to the *same object in memory* (identity, not value equality):

*Use == to compare values*

is tests memory identity. The only idiomatic use of is in Python is `if x is None:`

```
>>> a = 1
>>> b = 1
>>> a is b
True # works here: CPython caches small
 integers (-5 to 256)

>>> a = 1000
>>> b = 1000
>>> a is b
False # different objects even though
 values are equal!
```

## Exercise, I

- Assume that you go to a restaurant, and you order Rs.50 worth of food.
- Then you need to add the Sales Tax (8.875%) and add a tip (say, 20%).
- Write down the calculation that will print the total cost of the food.

## Exercise, II

- You have a stock that closed at Rs.550 on Monday, and then closed at Rs.560 on Tuesday.
- Calculate its daily return: the daily return is defined as the difference in the closing prices, divided by the closing price the day before.

## Exercise, III

- Write a Python program to solve  $(x + y) * (x + y)$
- Write a Python program to compute the future value of a specified principal amount, rate of interest, and a number of years. Test Data : amt = 10000, int = 3.5, years = 7. Expected Output : 12722.79
- Write a Python program to compute the distance between the points (x1, y1) and (x2, y2).

## Quick Check: Operators

### Think About It

10 / 3 and 10 // 3 both divide 10 by 3, but Python keeps them as two separate operators rather than one "divide" operator with a rounding flag. Why?

### Answer

Because they promise different things: / (true division) always returns a float – "here is the exact quotient" – while // (floor division) always returns a rounded-down whole value – "here is how many whole times this fits." Keeping them as distinct operators means you can tell the type of the result just by reading the code, instead of it depending on a runtime flag or the types of the operands.

